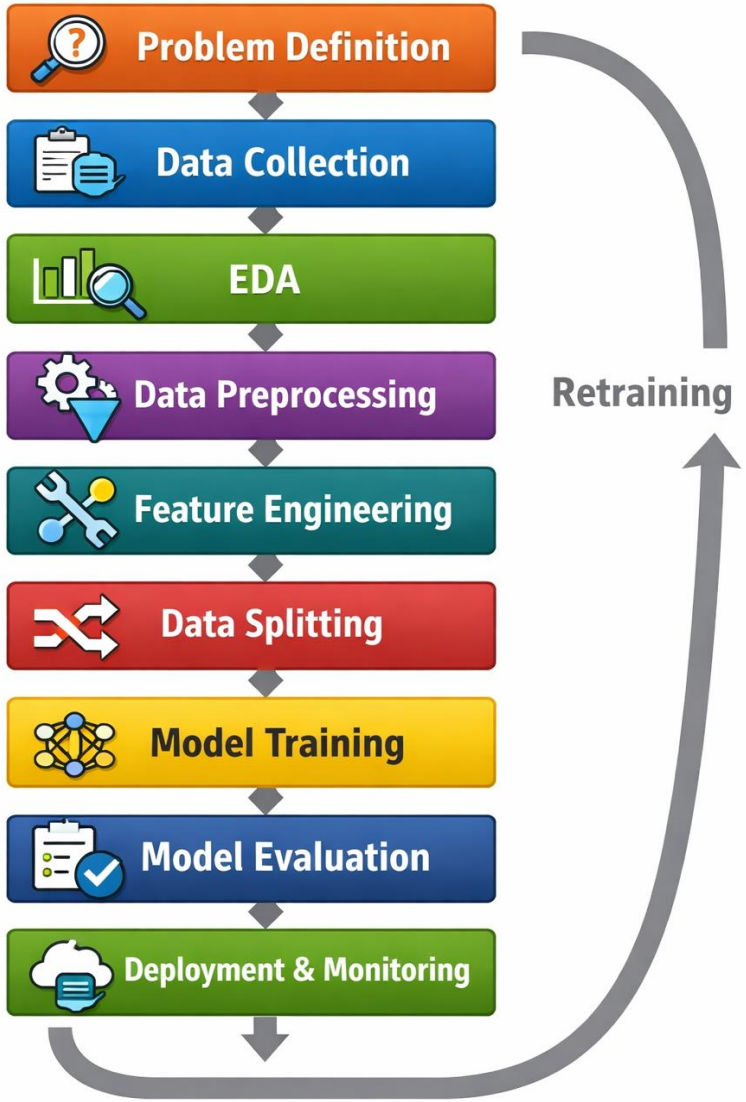
UNIT 1: Machine Learning Foundation
(Lecture 3)

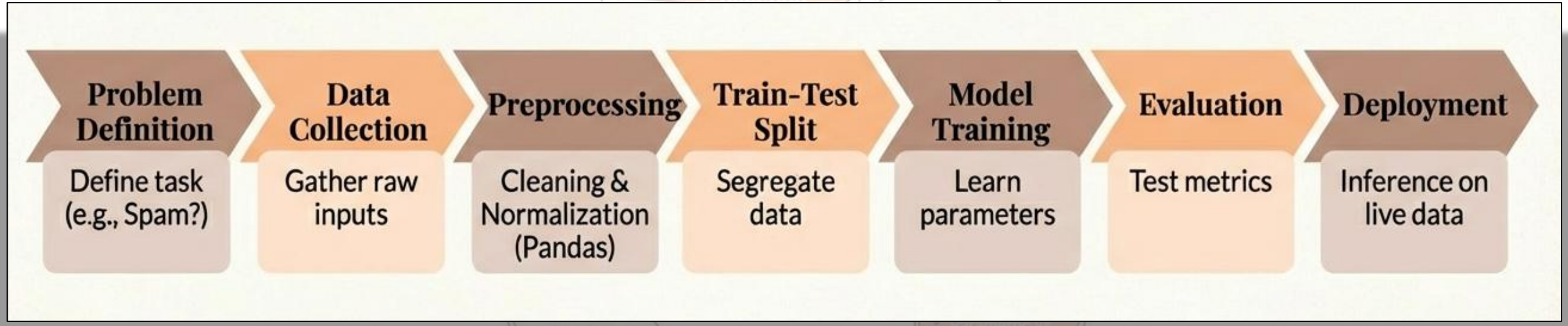
Machine Learning Workflow

What is Machine Learning Workflow?

- Step-by-step process to build ML systems
- Converts raw data into predictions
- Ensures accuracy, reliability, and scalability
- Iterative and continuous process


Problem Definition
Data Collection
EDA
Data Preprocessing
Feature Engineering
Data Splitting
Model Training
Model Evaluation
Deployment & Monitoring
Retraining

Based on Components of a Learning System



Step 1 – Problem Definition:

- Clearly define the problem
- Identify input features (X)
- Identify target variable (Y)
- Decide task type: Classification / Regression / Clustering
- Example: Predict house prices

Step 2 – Data Collection:

- Collect relevant and quality data
- Sources: databases, APIs, sensors, logs
- Data quantity and quality matter

Step 3 – Data Understanding & EDA:

- Understand data structure and types
- Identify missing values and outliers
- Visualize data patterns

Step 4 – Data Preprocessing:

- Handle missing values
- Remove duplicates and noise
- Encode categorical variables
- Normalize or standardize data

Step 5 – Feature Engineering:

- Select important features
- Create new meaningful features
- Reduce irrelevant information

Step 6 – Data Splitting:

- Split data into:
- Training set
- Validation set
- Test set
- **Use Case:** Prevents overfitting
- **Common split:** 70% / 15% / 15%

Training Set:

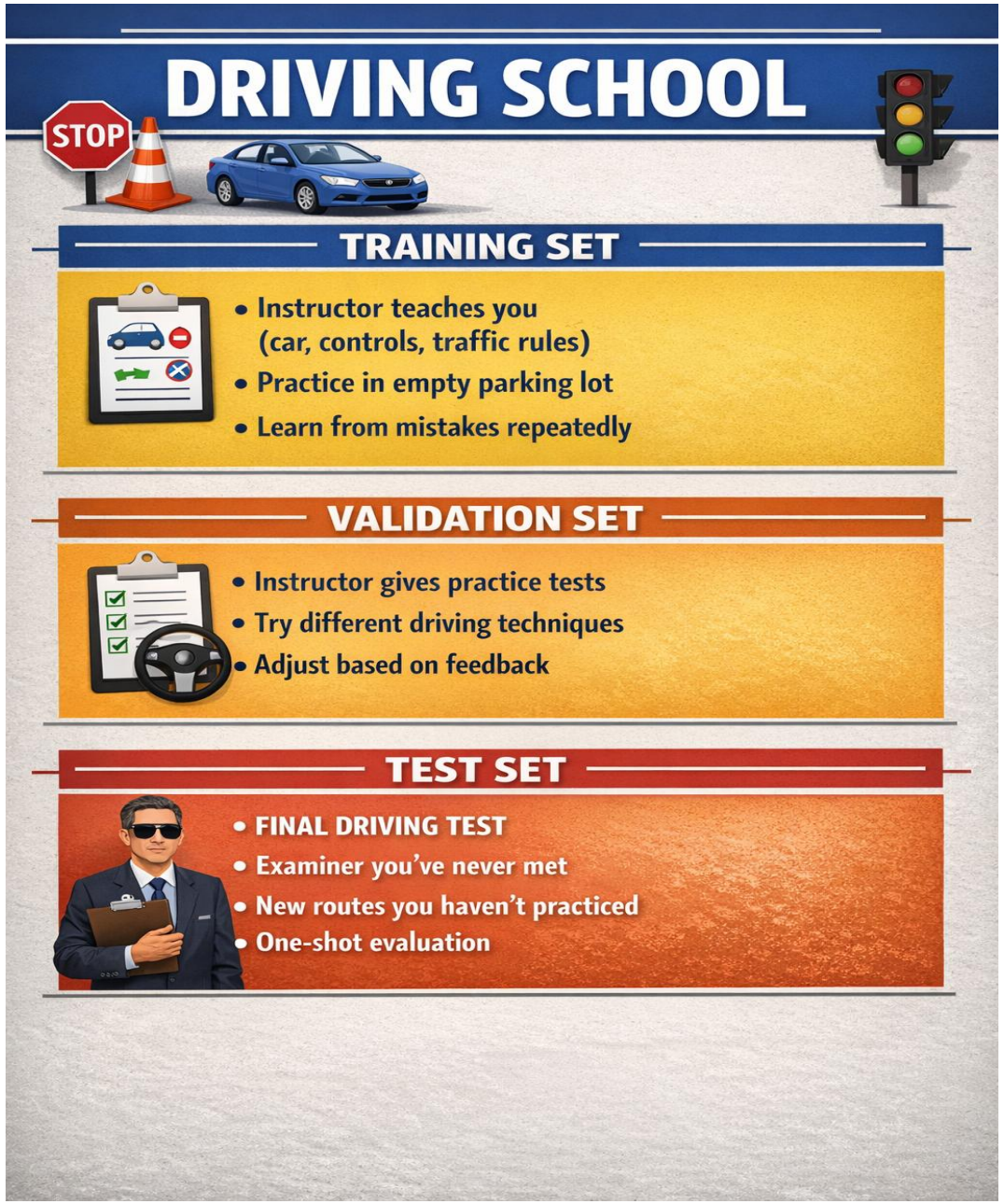
- Purpose: Teach the model
- Primary Role: Used to learn the patterns in the data
- How: Updates model weights/parameters through algorithms like gradient descent
- Analogy: Textbook for a student
- Example: 70% of your housing data used to learn price patterns

Validation Set:

- Purpose: Tune and Guide
- Primary Role: Hyperparameter tuning and model selection
- How: Evaluates different model configurations to choose the best one
- Analogy: Practice exams during study
- Example: 15% of data used to choose between different Random Forest depths

Test Set:

- **Purpose: Final Exam**
- **Primary Role: Unbiased evaluation** of final model performance
- **How:** Used **ONCE** at the end to simulate real-world performance
- **Analogy:** Final standardized test
- **Example:** 15% of data kept hidden until final model assessment


DRIVING SCHOOL
STOP
TRAINING SET
• Instructor teaches you (car, controls, traffic rules)
• Practice in empty parking lot
• Learn from mistakes repeatedly
VALIDATION SET
• Instructor gives practice tests
• Try different driving techniques
• Adjust based on feedback
TEST SET
• FINAL DRIVING TEST
• Examiner you've never met
• New routes you haven't practiced
• One-shot evaluation

Step 7 – Model Selection & Training:

- Choose suitable ML algorithm
- Train model using training data
- **Examples:**
- Linear Regression
- Logistic Regression
- Decision Tree
- Random Forest

Step 8 – Model Evaluation:

- Measure model performance
- Metrics depend on problem type
- **Examples:**
- Accuracy, Precision, Recall
- RMSE, MAE

Step 9 – Deployment & Monitoring:

- Deploy model to real-world system
- Monitor performance
- Retrain model with new data

What are Evaluation Metrics?

- Evaluation metrics are quantitative measures used to assess the performance of a model.
- They help compare models and understand strengths and weaknesses.

Why Evaluation Metrics Matter

- Measure model accuracy and reliability
- Help select the best model
- Identify overfitting or underfitting
- Support business and technical decisions

Classification Metrics

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

What is a Confusion Matrix?

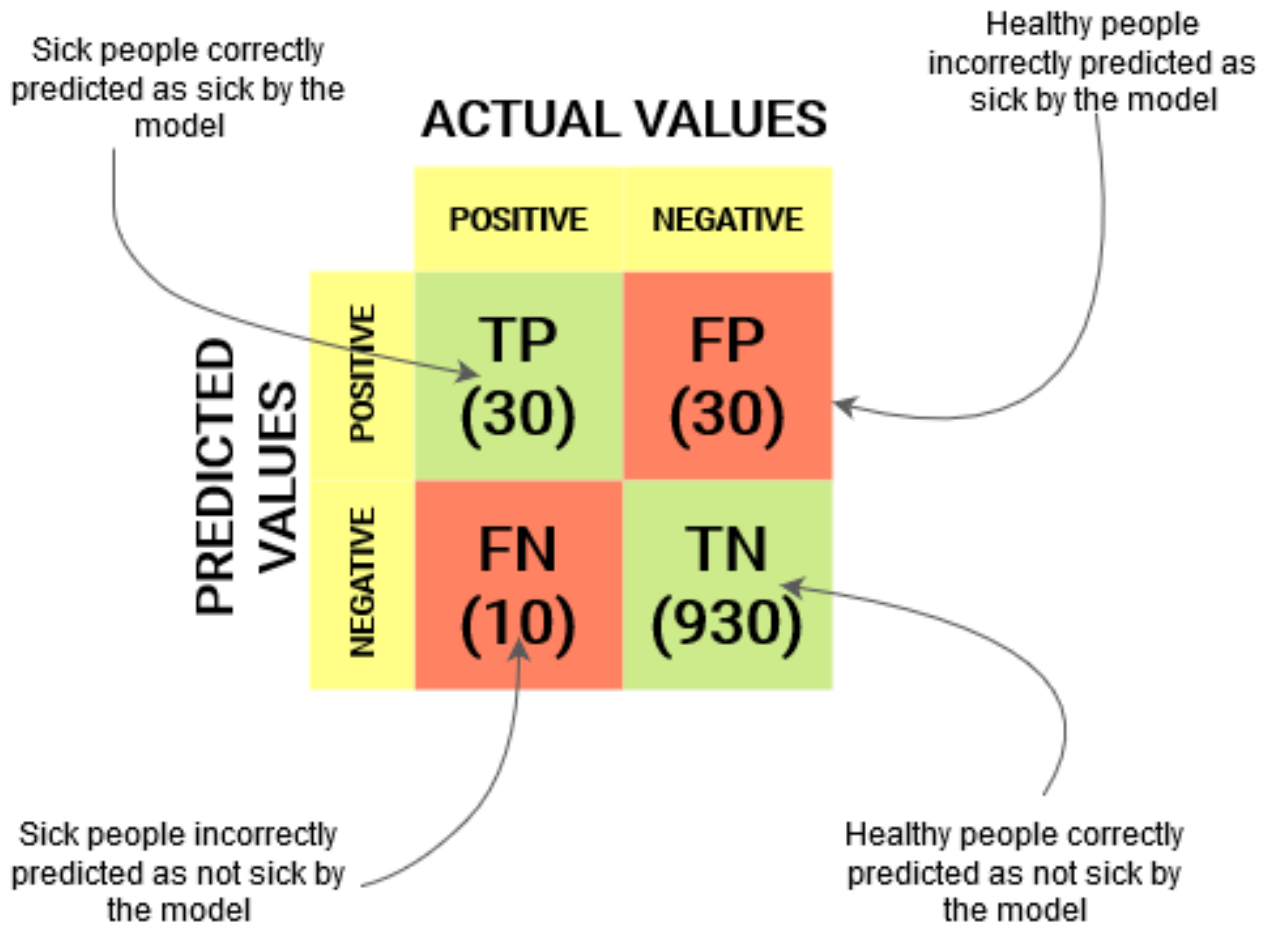
- A confusion matrix is a small table that helps you understand how well a classification model is making predictions.
- It compares:
- What the model predicted
- What the actual answer was

Actual \ Predicted	Spam	Not Spam
Spam	True Positive (TP)	False Negative (FN)
Not Spam	False Positive (FP)	True Negative (TN)

Meaning of Each Term:

- **True Positive (TP)**
  → Email is spam **and** model says spam
  (Correct)
- **True Negative (TN)**
  → Email is not spam **and** model says not spam
  (Correct)
- **False Positive (FP)**
  → Email is not spam but model says spam
  (False alarm)
- **False Negative (FN)**
  → Email is spam but model says not spam
  (Missed spam)

		Predicted class	
		P	N
Actual Class	P	True Positives (TP)	False Negatives (FN)
	N	False Positives (FP)	True Negatives (TN)



Task:

- Suppose we have 50 students' results.
  A model predicts whether a student will **Pass** or **Fail**.
- After testing, we get:
- 20 students **actually passed** and were predicted **Pass**
- 5 students **actually passed** but were predicted **Fail**
- 10 students **actually failed** but were predicted **Pass**
- 15 students **actually failed** and were predicted **Fail**

Term	Meaning	Value
TP	Actual Pass & Predicted Pass	20
FN	Actual Pass but Predicted Fail	5
FP	Actual Fail but Predicted Pass	10
TN	Actual Fail & Predicted Fail	15

Actual \ Predicted	Pass	Fail
Pass	TP = 20	FN = 5
Fail	FP = 10	TN = 15

Accuracy

- **How many predictions are correct overall?**
- Accuracy $= \frac{TP+TN}{TP+TN+FP+FN}$
- Accuracy = (Correct Predictions) / (Total Predictions)

- Best when classes are balanced.
- Can be misleading with imbalanced data.

Example:

Actual \ Predicted	Positive	Negative
Positive	TP = 40	FN = 10
Negative	FP = 5	TN = 45

Total Predictions

$$\text{Total} = TP + TN + FP + FN$$
$$= 40 + 45 + 5 + 10 = 100$$

How many predictions are correct overall?

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$= \frac{40 + 45}{100} = 0.85 = 85\%$$

Precision:

- **Out of predicted positives, how many are correct?**
- Precision $= \frac{TP}{TP+FP}$
- $= \frac{40}{40+5} = \frac{40}{45} = 0.89$

Recall:

- **Out of actual positives, how many did we catch?**
- Recall $= \frac{TP}{TP+FN}$
- $= \frac{40}{40+10} = \frac{40}{50} = 0.80$

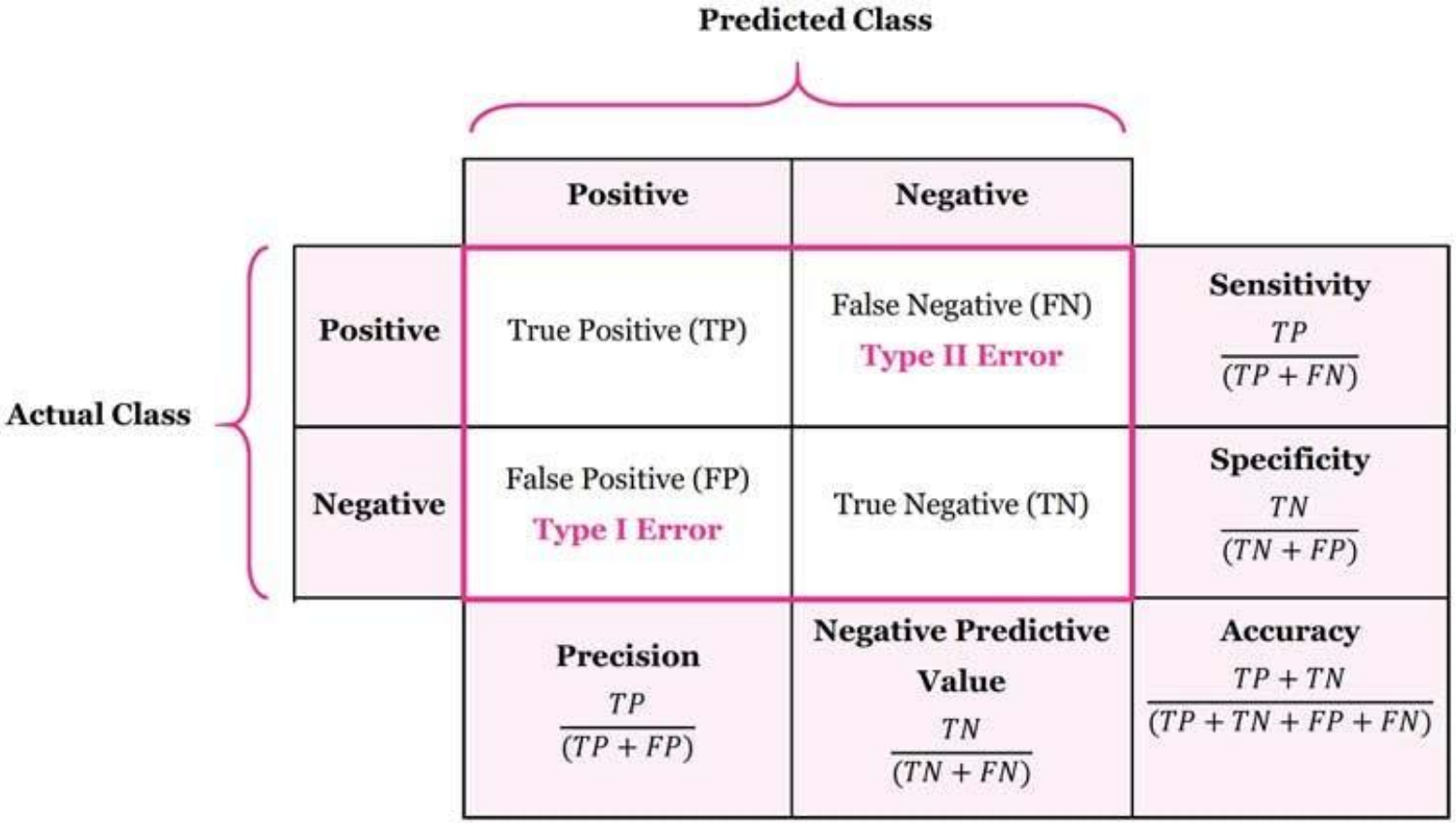
Precision & Recall:

- Precision: Correct positive predictions out of all positive predictions
- Recall: Correct positive predictions out of all actual positives

- Useful for imbalanced datasets.

F1-Score:

- F1-Score is the harmonic mean of Precision and Recall.
- Balances false positives and false negatives.
- **Balance between Precision and Recall**
- $F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$
- $= 2 \times \frac{0.89 \times 0.80}{0.89 + 0.80}$
- $= 2 \times \frac{0.712}{1.69} = 0.84$


Predicted Class
Positive
Negative
Actual Class
Positive
True Positive (TP)
False Negative (FN)
Type II Error
Sensitivity
$\frac{TP}{(TP + FN)}$
Negative
False Positive (FP)
Type I Error
True Negative (TN)
Specificity
$\frac{TN}{(TN + FP)}$
Precision
$\frac{TP}{(TP + FP)}$
Negative Predictive Value
$\frac{TN}{(TN + FN)}$
Accuracy
$\frac{TP + TN}{(TP + TN + FP + FN)}$

Task:

- A hospital uses a machine learning model to detect whether a patient has a disease.
- After testing the model on **200 patients**, the results are:
- **50 patients actually had the disease**
  - Model correctly identified **40** of them
  - Model missed **10** of them
- **150 patients did not have the disease**
  - Model wrongly predicted disease for **20** patients
  - Model correctly predicted no disease for **130** patients

Actual \ Predicted	Disease	No Disease
Disease	TP = 40	FN = 10
No Disease	FP = 20	TN = 130

ML Model Evaluation Metrics

Reference: Metric Cheat Sheet

Accuracy: (TP+TN) / Total	→ “Trustworthiness”
Precision: TP / (TP+FP)	→ “Coverage”
Recall (Sensitivity): TP / (TP+FN)	→ “True Negative Rate”
Specificity: TN / (TN+FP)	→ “True Negative Rate”
F1 Score: 2 * (Prec * Rec) / (Prec + Rec)	→ “Harmonic Mean”

ML Model Evaluation Metrics

Is Your AI a Good Sentry? A Guide to Machine Learning Metrics

The Sentry & The Wolf: An AI Analogy

Correct Calls

True Positive
(Threat Detected)

False Positive
(A "False Alarm")

True Negative
(Peaceful Night)

Critical Mistakes

False Negative
(A "Missed Danger")

How We Grade the Sentry (Key Metrics)

Precision
How often was the alarm real?

Prioritize this when the cost of a false alarm is high.

Recall
Did we catch every wolf?

Prioritize this when missing a single threat is disastrous.

Accuracy
What was the overall success rate?

Can be misleading if wolves (threats) are very rare.

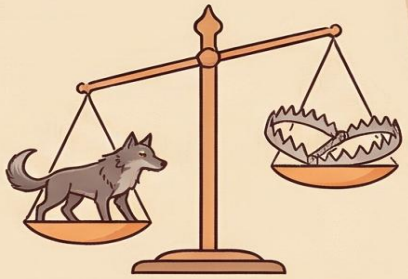
Professor's Challenge: Credit Card Fraud

Your model must detect credit card fraud.

Missing a fraud case is a disaster (big financial loss). A false alarm is a minor annoyance for the customer.

Which do you prioritize: **Precision** or **Recall?**

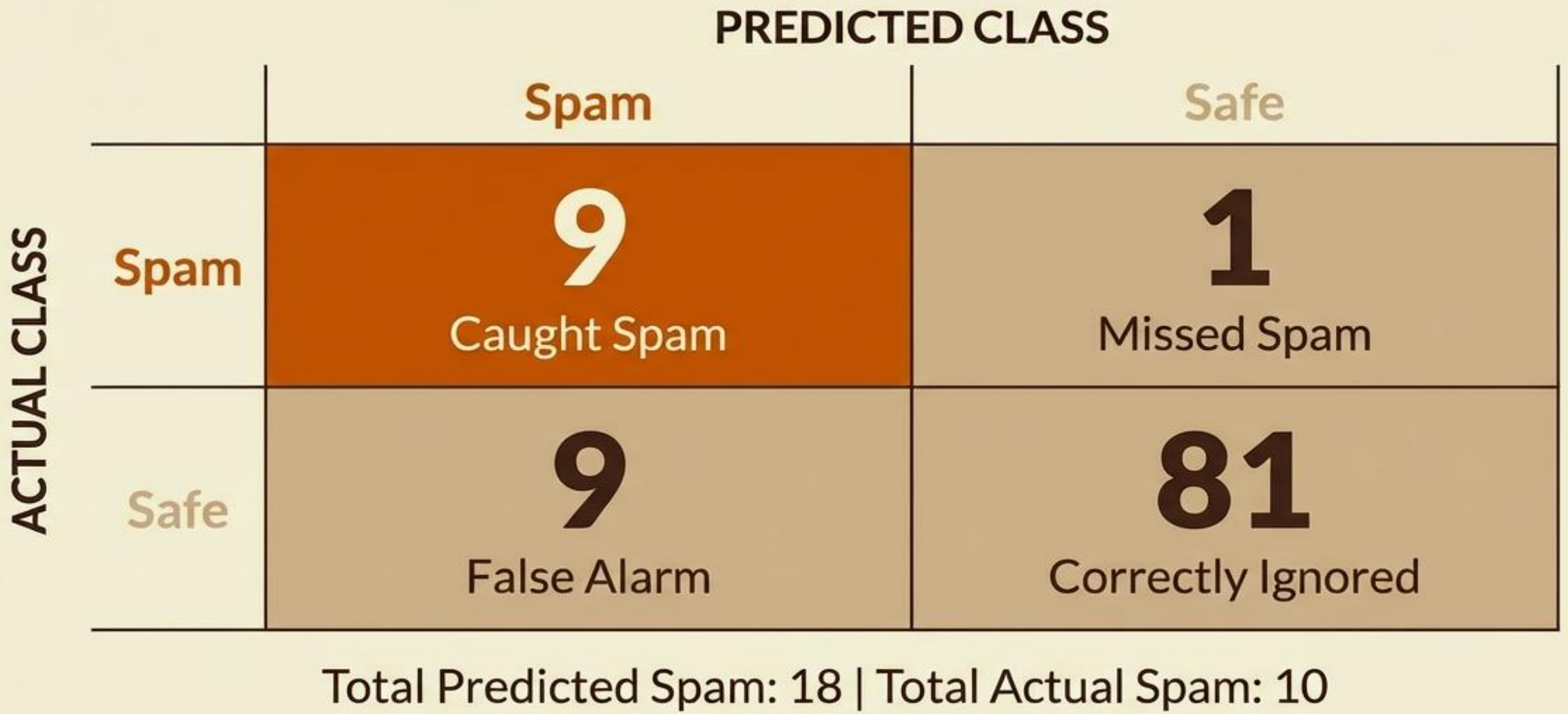
Recall



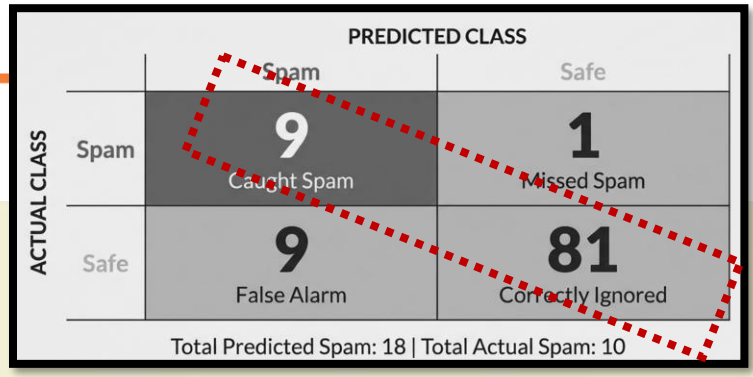
Recall. You must catch every potential "wolf" (fraud), even if it means more false alarms.

Scenario:

The Model's Performance Data



The Accuracy Trap



90%

Formula: (True Positives + True Negatives) / Total

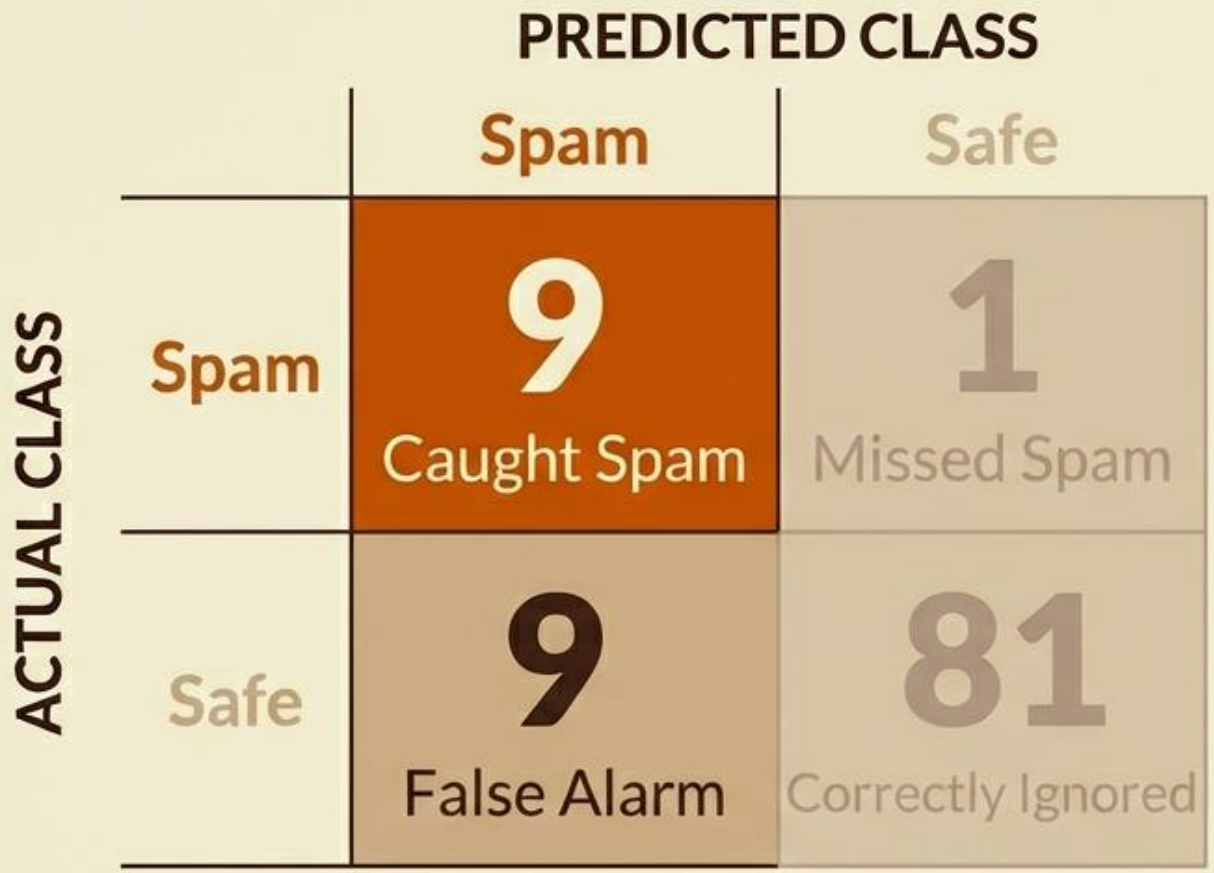
Calculation: (9 + 81) / 100 = 0.90

The Trap:

90% sounds excellent. But imagine a "lazy" model that simply marks EVERY email as Safe. Because 90% of our mail is safe, that lazy model would also score 90% accuracy.

Takeaway: Accuracy is deceptive on imbalanced datasets.

Precision: Can we trust the alarm?



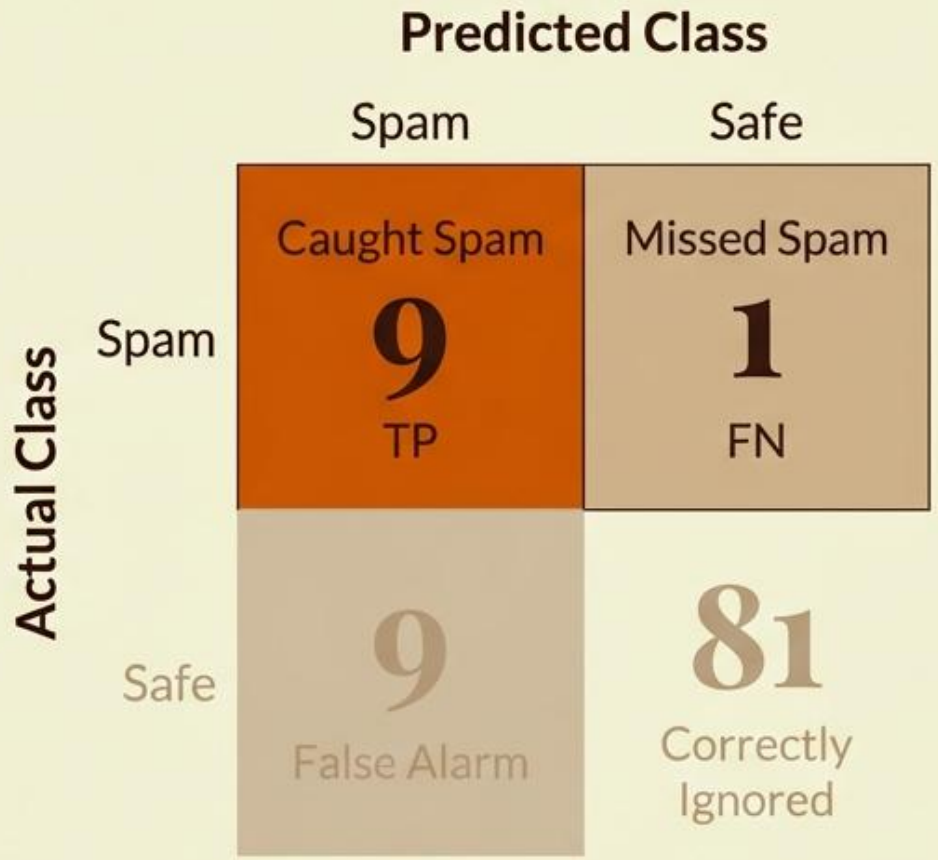
Definition: When the model screams 'SPAM', how often is it right?

Formula: TP / (TP + FP)

Calculation: 9 / (9 + 9) = 50%

Narrative:
Our model flagged 18 emails as spam, but was only right half the time. This means 50% of the alerts were false alarms, annoying the user.

Recall: Did we catch them all?



Definition: Out of all the actual spam, how much did we find?

Formula: TP / (TP + FN)

Calculation: 9 / (9 + 1) = 90%

Narrative: We found 90% of the spam. The model acts as an effective dragnet, letting very few threats slip through.

The Precision-Recall Trade-off

To increase **Precision** (avoid **false alarms**), we must be conservative.

To increase **Recall** (**catch all spam**), we must be aggressive.

Our Current Model: **Aggressive**. **High Recall** (90%) but **Low Precision** (50%).

F1 Score: The Harmonic Balance

The **F1 Score** is the **harmonic mean** of **Precision** and **Recall.** It penalizes extreme values, providing a single score that represents the balance.

$$F1 = 2 * \frac{(Precision * Recall)}{Precision + Recall}$$

$$2 * (0.5 * 0.9) / (0.5 + 0.9) = \mathbf{0.64}$$

Verdict: While Accuracy was 90%, the F1 Score of 0.64 reveals the model is actually mediocre because of the poor Precision.

Specificity: The True Negative Rate

	Predicted Spam	Predicted Safe
Actual Spam	91	9
Actual Safe	9 FP	81 TN

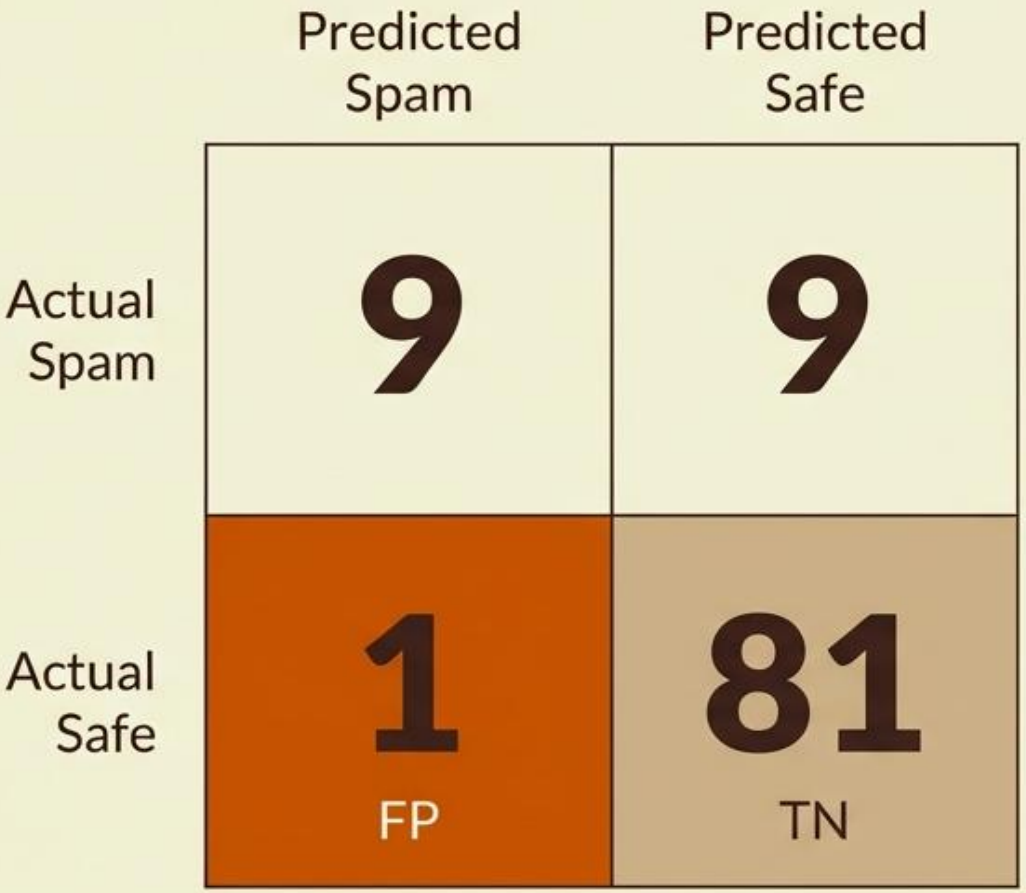
Definition: How effective is the model at leaving safe emails alone?

Formula: TN / (TN + FP)

Calculation: 81 / (81 + 9) = **90%**

Context: 90% Specificity means 10% of safe emails are being wrongly flagged. In a business context, losing 1 in 10 legitimate emails is a major problem.

Model Performance Report



Accuracy: 90% (Misleading)

Precision: 50% (Poor)

Recall: 90% (Strong)

Specificity: 90% (Strong)

F1 Score: 0.64 (Moderate)

This model is safe for a 'Junk' folder, but too aggressive to delete emails automatically.

Choosing the Right Metric

Scenario A: Spam Filter

Cost of False Positive is High (Lost Mail)

Goal: High Precision

We accept some spam to protect real mail.

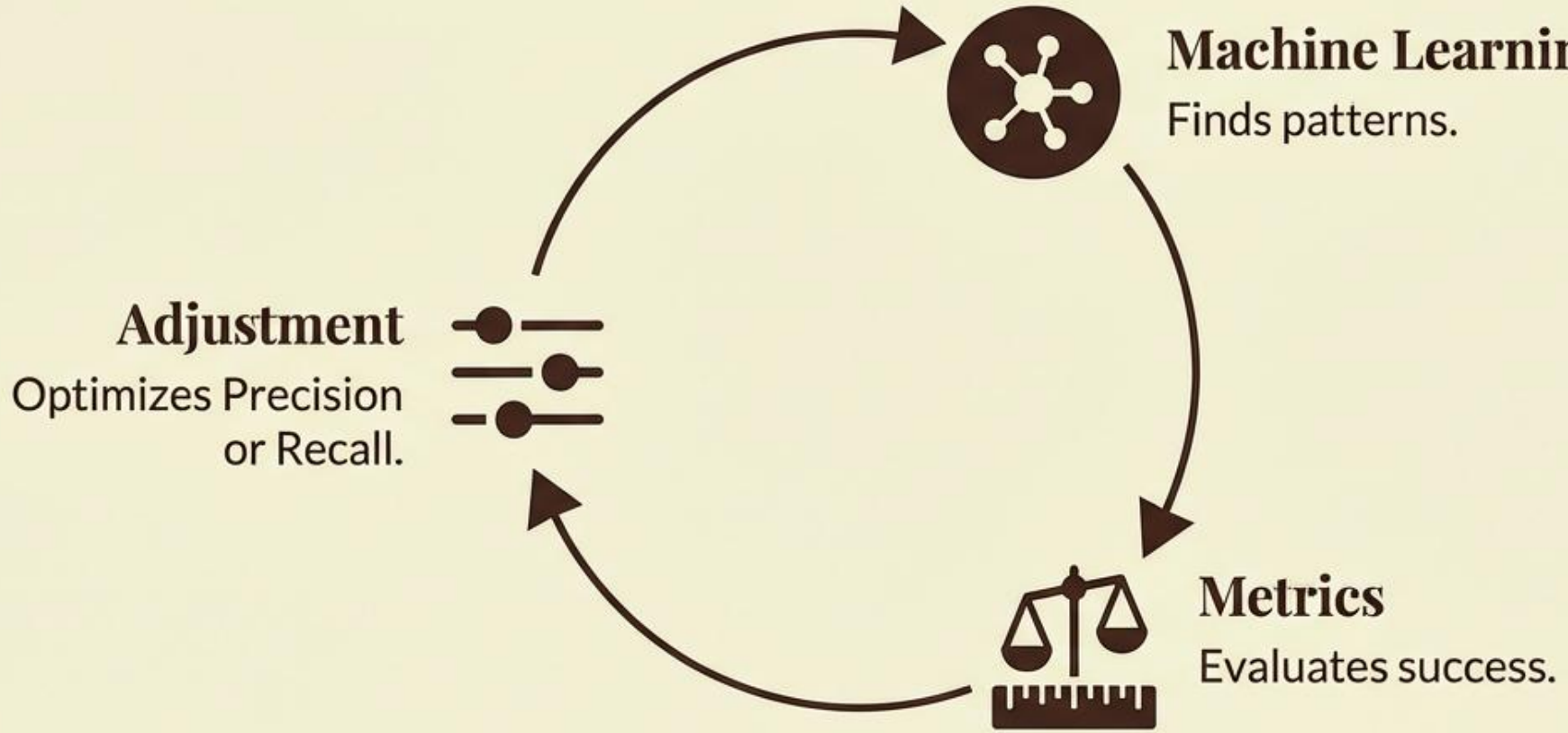
Scenario B: Disease Detection

Cost of False Negative is High (Missed Diagnosis)

Goal: High Recall

We accept false alarms to ensure we don't miss a patient.

The Loop of Improvement



A model with 99% accuracy can be useless if it fails the specific metric that matters to to your users. Always look beneath the top-line number.

Regression Metrics

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R-squared

MAE, MSE, RMSE:

- MAE: Average absolute errors
- MSE: Average squared errors
- RMSE: Square root of MSE

- Lower values indicate better performance.

R-squared:

- Represents how well the model explains variance in data.
- Ranges from 0 to 1.
- Higher is better.

Evaluation Metrics for Regression Models

- When we are predicting a continuous number (like house prices), we measure how far off our arrows (predictions) are from the bullseye (expected result).

Metric	Description	Key Characteristics	Error Sensitivity
MAE (Mean Absolute Error)	Average of the absolute errors.	Treat all errors equally; robust to outliers.	Low; it does not disproportionately penalize large errors.
MSE (Mean Squared Error)	Average of the squared errors.	Penalizes large errors heavily; very useful for optimization.	High; large errors are squared, making them much more significant.
RMSE (Root MSE)	The square root of MSE.	Returns the error value to the same units as our data (e.g., "Dollars").	High; like MSE, it is sensitive to large errors due to the squaring process.
R^2 Score	The proportion of variance explained.	Tells you how much better your model is than just guessing the average.	N/A; measures fit quality relative to a baseline mean model.

The Target Range: Five Houses, Five Predictions

To evaluate our model, we track 5 distinct properties. This 'Ledger' will be the foundation for every metric calculation.

Property ID	Actual Price (Target)	Predicted Price (Arrow)	Observation
House A	$300,000	$310,000	Small Overestimation
House B	$450,000	$440,000	Small Underestimation
House C	$200,000	$200,000	Perfect Shot
House D	$600,000	$550,000	**SIGNIFICANT MISS**
House E	$350,000	$360,000	Small Overestimation

Note House D. The model underestimated by $50k. This outlier is the critical test for our metrics.

Measuring the Distance (The Residual)

The 'Error' is simply the mathematical distance between the Arrow and the Bullseye.

Error = Actual Value - Predicted Value

House	Actual	Predicted	Calculation	Error (Residual)
A	$300k	$310k	300 - 310	-$10k
B	$450k	$440k	450 - 440	+$10k
C	$200k	$200k	200 - 200	$0
D	$600k	$550k	600 - 550	+$50k
E	$350k	$360k	350 - 360	-$10k

Problem: If we simply sum these errors (-10 + 10 + 0 + 50 – 10), the negatives cancel the positives. We need a way to summarize the magnitude of the error, not just the direction.

Mean Absolute Error (MAE):

The Logic

Definition: The average of the absolute errors.

Characteristics: Treats all errors equally. It is robust to outliers and gives a realistic view of the "average" miss.

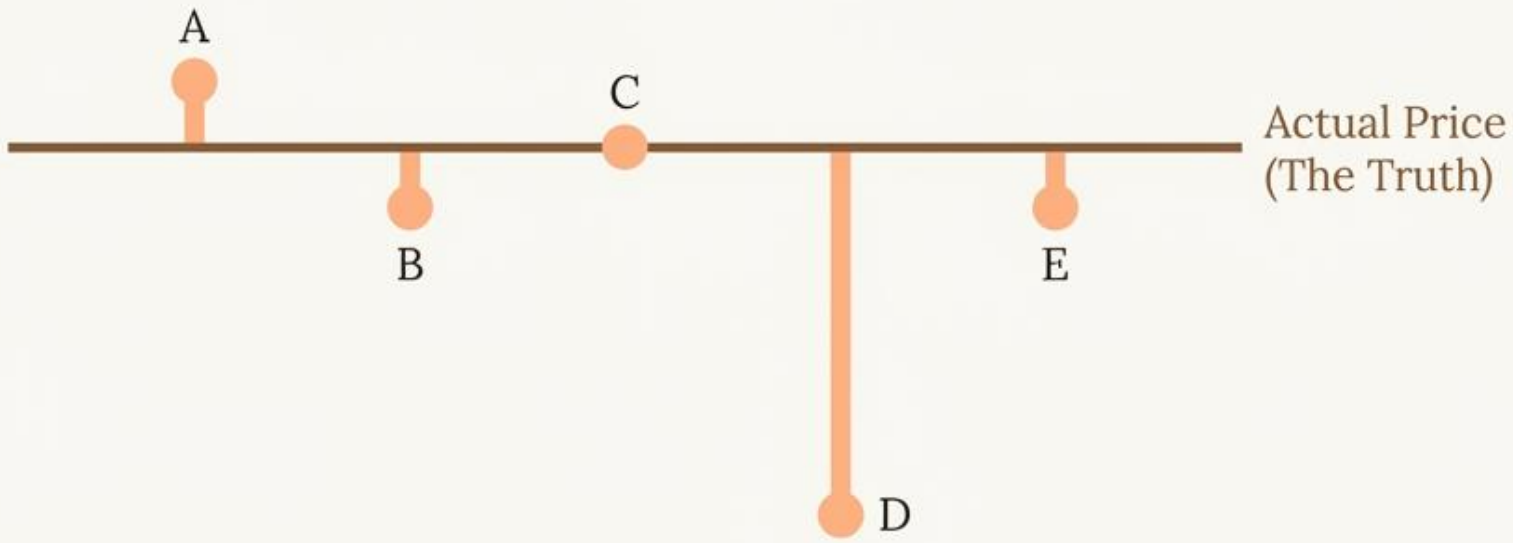
$$\text{MAE} = \frac{(\text{Sum of } |\text{Absolute Errors}|)}{\text{Number of Observations}}$$

The Ledger

House A:	$\lvert -10 \rvert$	→	10
House B:	$\lvert +10 \rvert$	→	10
House C:	$\lvert 0 \rvert$	→	0
House D:	$\lvert +50 \rvert$	→	50
House E:	$\lvert -10 \rvert$	→	10
	Sum:		80
	Average (80 ÷ 5):		16

MAE = $16,000. On average, our Archer misses the price by $16k.

Visualizing the Absolute Error



MAE measures the average length of these orange bars. It ignores whether the bar goes up or down—it only cares about the length.

Mean Squared Error (MSE):

The Logic

Definition: The average of the squared errors.

Characteristics: Penalizes large errors heavily. By squaring the error, a big miss becomes a massive number. Useful for optimization algorithms.

$$\text{MSE} = \frac{(\text{Sum of Errors}^2)}{\text{Number of Observations}}$$

The Ledger

House A:	$10^2 \rightarrow$ 100
House B:	$10^2 \rightarrow$ 100
House C:	$0^2 \rightarrow$ 0
House D:	**$50^2 \rightarrow$ 2,500**
House E:	$10^2 \rightarrow$ 100
Sum:	2,800
Average (2800 ÷ 5):	560

MSE = 560. Note: The unit is now "Dollars Squared" ($\$^2$). Hard to interpret, but mathematically powerful.

Root Mean Squared Error (RMSE):

We take the Square Root to bring the units back to reality.

MSE (Dollars Squared)
560

Square Root √

RMSE (Dollars)
23.66

The Comparison

MAE: $16k

The average absolute miss.

RMSE: $23.66k

The error penalized for the large outlier.

Why is RMSE higher? Because House D's large $50k miss was squared, dragging the average up. RMSE is screaming that we have an outlier problem.

Choosing Your Scorecard: MAE vs. RMSE

MAE (The Steady Hand)

Behavior: Robust to outliers.

Use Case: Pricing mass-market condos. Being off by $50k on one strange luxury house shouldn't panic the whole model. You want the general median accuracy.

RMSE (The Perfectionist)

Behavior: Sensitive to outliers.

Use Case: Pricing high-risk assets. A single massive miss could be catastrophic (like bank fraud or structural failure). You need a metric that reacts strongly to large errors.

R^2 Score:

- R^2 tells you how much better your model is compared to just guessing the average.
- **Step 1: The laziest prediction**
- Imagine you ignore all inputs and always predict the average value.
- That's your **baseline**.
- **Step 2: Your regression model**
- Now your model uses features (size, location, etc.) to make smarter predictions.

👉 **R^2 compares your model against that lazy baseline.**

Formula

$$R^2 = 1 - \frac{Model\ Error}{Baseline\ Error}$$

Where:

- **Model Error** = how wrong your model is
- **Baseline Error** = how wrong the "predict-the-mean" approach is

What Different R^2 Values Mean:

$R^2 = 1$

→ Model has **zero error**

→ Perfect predictions

$R^2 = 0$

→ Model is **no better than predicting the average**

$R^2 = 0.7$

→ Model removes **70% of the error**

→ Only **30% is left unexplained**

$R^2 < 0$

→ Model is **worse than guessing the average**

Choosing the Right Metric:

- Depends on problem type
- Consider data balance
- Understand business impact of errors
- Use multiple metrics when possible